

Sphatik OS — Build Plan

Sep 28, 2026 · @Sarvesh Kumar Yadav

How the build is ordered

Sphatik is built in eight stages, from the bottom of the system up: foundations first, then pixels, then the system that feeds them, then the experience, then everything that runs on top. Each stage is broken into numbered work packages (for example 2.3), and each work package into tasks in the issue tracker.

Eight stages, each built on the ones above it

Green pills are gates: the next dependent stage starts in earnest only when the gate passes

1	Foundations Repo, CI, sp tool, toolchain; phone unlocked on the postmarketOS base	Bring-up done Months 0–2
2	Graphics core on the laptop Compositor skeleton, GLES renderer, glass pipeline, springs, gestures	Glass on laptop Months 1–4
3	Graphics on the phone DRM/KMS backend, touch input, 60 fps glass on the Adreno 618	Glass on phone Months 4–6
4	Toolkit and system services sphatik-ui, sphatikd, modem, audio, Wi-Fi and Bluetooth wired over D-Bus	Months 5–9
5	Shell and phone basics Lock, home, panels, Halo, switcher; Phone, Messages, Settings, Camera, Clock	Usable phone Months 7–11
6	App platform, privacy and signature features Sandbox, permissions, .spk and SDK, other stock apps, privacy v1, v1 features	Months 10–15
7	Android layer and security Container with microG, encryption, verified A/B images, updates, own init	Android apps Months 13–18
8	Beta and release Hardening, performance budgets, docs, signed staged rollout	Public beta Months 18–20

Sphatik build order · 8 stages, 6 gates, about 20 months part-time

Why this order

1. **Hardware and tools before code.** Nothing matters until the phone boots mainline Linux and the laptop builds for it.

2. **Pixels before features.** The glass compositor is the riskiest and most visible part; prove it early, first on the laptop, then on the phone.
3. **A usable phone before a clever phone.** Calls, SMS, Wi-Fi and notifications must be reliable before the platform, privacy tooling and signature features.
4. **Permissions before third-party apps.** The sandbox and permission broker exist before any app that is not ours can be installed.
5. **Android last among the big pieces.** It depends on a stable compositor, services and security model, and it is the most memory-hungry.

Hierarchy used in this plan: Stage → Work package → Task. Stages and work packages live here; tasks live in the issue tracker, labelled with their work package ID. The estimate assumes one developer working part-time (about 15 hours a week); a second contributor shortens it by roughly a third.

Before you start

Five things only you can do, which unblock everything else. Start the first one today, because it involves waiting.

- ☐ **Request the bootloader unlock** with Xiaomi's Mi Unlock tool. It enforces a waiting period, so let the clock run while you do laptop work.
- ☐ **Back up the Redmi Note 9 Pro** (photos, chats, files) and switch to a **spare phone** for daily use; the Redmi will be wiped and unreliable for months.
- ☐ **Install Linux natively** on your laptop (Ubuntu 24.04 or Arch), not in a virtual machine, so the compositor can use the real GPU.
- ☐ **Create the GitHub repository** for Sphatik and commit the docs zip as the first commit.
- ☐ **Block regular time:** about 15 hours a week, in fixed slots, with a weekly 30-minute review against this plan.

Skills to brush up on in parallel (a few evenings each): Rust ownership and async basics; how Wayland compositors work (the Smithay examples); OpenGL ES shaders; Linux boot and device trees at a reading level.

Stage 1: Foundations

Goal: a repository that builds for the laptop and the phone, and a Redmi that boots mainline Linux with every hardware feature tested. Months 0 to 2.

ID	Work package	Needs	Done when
1.1	Monorepo and Cargo workspace with empty crates: <code>sphatik-comp</code> , <code>sphatik-shell</code> , <code>sphatik-ui</code> , <code>sphatikd</code> , <code>sp</code>	Before you start	<code>cargo build --workspace</code> passes
1.2	CI: format, lint, tests, aarch64 cross-build	1.1	Green check on a pull request
1.3	<code>sp</code> developer tool skeleton: <code>build</code> , <code>logs</code> , <code>flash</code>	1.1	<code>sp --help</code> lists the commands
1.4	Laptop toolchain: Rust, cargo-zigbuild, Mesa and libinput dev packages, pmbootstrap	Before you start	Hello-world cross-compiles and runs on the phone over SSH
1.5	Device bring-up: stock firmware, unlock, postmarketOS mlatoll image with Phosh	Unlock approved	Phone boots to Phosh
1.6	Hardware checklist and baselines in <code>STATUS.md</code>	1.5	Every row filled; idle RAM, boot time and <code>glmark2-es2</code> score recorded
1.7	SSH over USB and log streaming	1.5	<code>sp logs</code> shows live logs from the phone

Gate: Bring-up done. Base image boots on curtana, the checklist is recorded, and the workspace cross-compiles. If calls or audio fail on the base image, fix or report them upstream now; a phone that cannot call never becomes a daily driver.

Stage 2: Graphics core on the laptop

Goal: the compositor runs in a laptop window, draws the Liquid Aurora wallpaper and real glass, and moves on springs driven by drag gestures. Months 1 to 4, overlapping Stage 1 because it needs only the laptop.

ID	Work package	Needs	Done when
2.1	Smithay winit backend: window, event loop, 60 fps frame clock	1.1	Steady frame timer in a window

ID	Work package	Needs	Done when
2.2	GL ES renderer: textured quads, rounded rectangles, gradients, damage tracking	2.1	Wallpaper renders; unchanged frames cost almost nothing
2.3	Wallpaper engine: port Liquid Aurora, Obsidian and crystal art to GPU-rendered vector shapes	2.2	Matches the prototype side by side
2.4	Glass pipeline v0: backdrop copy, quarter-resolution dual Kawase blur, upsample	2.2	A blurred rounded panel over the wallpaper
2.5	Glass pipeline v1: SDF refraction, tint, rim light, specular, dispersion, blur cache	2.4	Matches prototype glass; under 3 ms on the laptop
2.6	Spring engine crate with the four presets and velocity hand-off	1.1	Unit tests match the design spec values
2.7	Input and gesture recogniser: edge zones, swipe up, pull down, interrupt any animation	2.1, 2.6	Panel follows the drag and springs back at 60 fps
2.8	Wayland client support: xdg-shell, linux-dmabuf; show an external app full screen	2.2	A terminal or test app renders inside
2.9	Debug overlay: FPS, frame graph, damage flashes, glass cost	2.2	Toggle with F12

Gate: Glass on laptop. Lock screen mock-up with wallpaper, clock text and one glass panel runs at 60 fps in a window, and swipe gestures feel as good as the prototype.

Stage 3: Graphics on the phone

Goal: the same compositor runs full screen on the Redmi, replacing Phosh, at 60 fps with real touch. Months 4 to 6.

ID	Work package	Needs	Done when
3.1	DRM/KMS + GBM + EGL backend on Freedreno; session through libseat	2.2, 1.6	Wallpaper on the phone screen, no tearing

ID	Work package	Needs	Done when
3.2	Touch through libinput: multi-touch, edge zones scaled to the panel	3.1, 2.7	Swipe gestures work on the glass
3.3	Fractional scale 2.75 and the 393-point logical canvas	3.1	Layout matches the prototype on the real panel
3.4	Performance pass on Adreno 618: glass tiers, blur cache, CPU pinning to the big cores	3.1, 2.5	Zero dropped frames in the gesture test; glass under 3 ms GPU
3.5	Replace Phosh with sphatik-comp as the session; crash restart within 1 second	3.1	Phone boots straight into Sphatik's compositor
3.6	Sensors for tilt light and parallax (accelerometer through iio)	3.1	Specular spot and wallpaper follow tilt

Gate: Glass on phone. The Stage 2 lock screen runs full screen on curtana at 60 fps, touch works everywhere, and glass stays under its 3 ms budget. This is the moment to record a demo video and invite contributors.

Stage 4: Toolkit and system services

Goal: the building blocks every screen and app needs: the `sphatik-ui` toolkit and the `sphatikd` system services, with the phone's hardware reachable over D-Bus. Months 5 to 9. Two tracks run in parallel: toolkit (laptop) and services (phone).

Track A: sphatik-ui toolkit

ID	Work package	Needs	Done when
4.1	Reactive state (signals) and declarative view tree	1.1	A counter app updates only what changed
4.2	Layout with taffy; design tokens and theming, light and dark	4.1	Sample screens match the spec's spacing and colours
4.3	Text with cosmic-text: shaping, Devanagari, text styles, 80% to 310% sizes	4.2	Hindi and English render crisply at every size

ID	Work package	Needs	Done when
4.4	Component library, first half: title bar, tab bar, list row, grouped card, button, toggle, search field, segmented control	4.2, 4.3	Golden images match the prototype
4.5	Component library, second half: sheet, menu, toast, banner, slider, text field, avatar, action button, pushed view	4.4	Golden images match
4.6	Accessibility tree (AccessKit) and Fluent strings	4.4	Every component exposes a label and role
4.7	sphatik_glass_v1 client side: apps request glass regions	4.4, 2.5	An app shows real compositor glass behind its tab bar

Track B: sphatikd and hardware services

ID	Work package	Needs	Done when
4.8	sphatikd skeleton on D-Bus with Settings and Power	1.7	Settings survive a reboot; battery level readable
4.9	Notifications interface with channels and importance levels	4.8	A test app posts and withdraws notifications
4.10	Apps registry and launcher, LiveActivities registry	4.8	Launch an app by ID; start and end a live activity
4.11	Telephony: ModemManager, Qualcomm helpers, callaudiod	1.6	Place and receive a call from a command line
4.12	Audio (PipeWire), Wi-Fi (iwd), Bluetooth (BlueZ), location (Geoclue)	1.6	Each controllable over D-Bus from a test client
4.13	Suspend policy and wake sources (modem, RTC, power button)	4.8, 4.11	Screen-off suspend; a call wakes the phone

No gate at the end of Stage 4: it feeds Stage 5 directly, which ends with the Usable phone gate.

Stage 5: Shell and phone basics

Goal: the Sphatik experience from the prototype, running on the phone, good enough to replace a normal phone for a week. Months 7 to 11.

ID	Work package	Needs	Done when
5.1	Boot splash (SKY) hand-off from initramfs to compositor	3.5	SKY centred on boot, cross-fades to the lock screen
5.2	Lock screen: clock, complications, notifications, PIN pad, customisation	4.5, 4.9	Unlock with PIN; customisation persists
5.3	Home: grid, widgets, dock, edit mode, App Library	4.5, 4.10	Rearrange icons, add a widget, open the library
5.4	Notification Center and banners	4.9	Real notifications group, dismiss and reply
5.5	Control Center: Wi-Fi, data, Bluetooth, sliders, media, tiles	4.12	Every toggle changes the real hardware state
5.6	Halo with call, timer and media activities	4.10	A call shows in the Halo with Mute and End
5.7	App switcher and gesture navigation (home, back, quick flip)	3.2	Switch between three apps without a stutter
5.8	Intent Bar: apps, settings, sums	4.10	Finds apps and settings, answers sums
5.9	Phone app with in-call screen	4.11, 5.6	Calls in and out with a working in-call UI
5.10	Messages app for SMS	4.11	Send and receive SMS, including OTPs
5.11	Settings app, first pages: network, display, sound, battery	4.8, 4.12	Common settings changeable without a terminal
5.12	Camera app (viewfinder, photo) and Clock app	1.6 camera status	Take a photo; alarms wake the phone

Gate: Usable phone. Calls, SMS, mobile data, Wi-Fi, audio, notifications, camera and alarms work for **a full week of daily use** without falling back to the spare phone. Stop new feature work until this passes.

Stage 6: App platform, privacy and signature features

Goal: Sphatik becomes a platform: third-party apps can be installed safely, the rest of the stock apps arrive, the v1 privacy system is live, and the v1 signature features ship. Months 10 to 15.

Order inside this stage matters: 6.1 to 6.3 (sandbox and permissions) come before anything that installs apps.

ID	Work package	Needs	Done when
6.1	App sandbox: per-app user, namespaces, seccomp, Landlock	4.10	A test app cannot read another app's files or the network
6.2	Permission broker, system prompts, time-boxed grants, auto-revoke, indicators and kill switches	6.1, 4.8	Default-deny works for camera, mic, location, internet
6.3	Portals: file picker, camera and mic, location, share sheet	6.2	Sandboxed apps reach data only through portals
6.4	.spk format, signing, installer; SDK crates and <code>sp new / sp package</code>	6.1, 4.5	Install a signed third-party app; updates must match its key
6.5	Privacy v1: per-app internet firewall, random identifiers, paste-only clipboard, 30-day Privacy Ledger, tracker blocking, Privacy Center	6.2	Ledger records every access; trackers blocked system-wide
6.6	Remaining stock apps: Gallery, Files, Browser, Calendar, Notes, Mail, Music, Weather, Maps, Calculator, Store, Terminal	6.3, 6.4	All 17 apps usable, matching the prototype
6.7	Signature v1: Halo Shelf, living wallpapers, Hindi for the whole phone, Longevity settings (80% charge limit, keep fast)	5.3, 5.6	Each works on the phone as in the prototype
6.8	Keyboard: sphatik-keyboard with swipe typing, Hindi and Hinglish	4.3	Type in every app, including OTP autofill

No gate at the end of Stage 6; its exit check is that every stock app and v1 feature passes its UI flow test on the device.

Stage 7: Android layer and security

Goal: Android apps run safely alongside native ones, and the system becomes trustworthy: encrypted, verified, updatable and rolled back automatically if an update fails. Months 13 to 18. Two tracks in parallel.

Track A: Android compatibility

ID	Work package	Needs	Done when
7.1	Kernel options for binderfs, namespaces and cgroups verified on curtana	1.6	Container prerequisites pass a self-test
7.2	Android image recipe: LineageOS arm64 with microG signature spoofing, Mesa and minigbm	7.1	Image boots headless in the container
7.3	Container manager: start on demand, freeze when idle, stop under memory pressure	7.2, 4.10	First Android app opens in a few seconds; idle container uses no CPU
7.4	Multi-window integration: each Android app a Sphatik window with gestures and switcher cards	7.3, 5.7	An Android app behaves like a native one in the switcher
7.5	sphatik-bridge APK: notifications, clipboard, shared folder, icons in the App Library	7.3, 4.9	WhatsApp notifications appear in Notification Center with reply
7.6	Host permissions, firewall and Ledger applied to Android apps	6.5, 7.3	Android apps show up in the Ledger and firewall
7.7	Setup flow: download image, install microG, offer Aurora Store and F-Droid	7.5	A new user reaches a working WhatsApp in one guided flow

Track B: security and updates

ID	Work package	Needs	Done when
7.8	Credential-encrypted home (fscrypt, Argon2id from the PIN) and device-encrypted area	5.2	Data unreadable after reboot until the PIN is entered
7.9	EROFS system image, dm-verity, signing keys, initramfs verification	1.3	Tampered images refuse to boot

ID	Work package	Needs	Done when
7.10	A/B slots, boot counter, automatic rollback, delta updates, update service in sphatikd	7.9	A deliberately broken update rolls back by itself
7.11	sphatik-init replaces the base init; service files, cgroups, crash restart	7.9	Boot to lock screen under 15 s with sphatik-init
7.12	Lockdown, USB data lock while locked, auto-reboot after idle	7.8	All three work from the lock screen and power menu

Gate: Android apps. WhatsApp installed through Aurora Store sends and receives messages, and its notifications appear in Notification Center; signed updates install and roll back correctly.

Stage 8: Beta and release

Goal: turn a working system into something other people can install and trust. Months 18 to 20. v2 features (Desktop mode, Glass Lens, Offline Nearby, Private Memory, decoy data, domain firewall, Private Space) come after the beta, in monthly releases.

ID	Work package	Needs	Done when
8.1	Performance hardening against every budget in the test plan	All stages	Two consecutive nightly runs meet every budget
8.2	Soak and battery tests: 8-hour standby, interaction loop, memory growth	8.1	Standby under 1% per hour; no memory growth
8.3	Accessibility and localisation pass: screen reader, 310% text, Reduce Motion and Transparency, Hindi	6.6	Checklist in the HIG passes for every stock app
8.4	Installer images, install guide, recovery guide, known-limitations page	7.10	A tester installs from the guide alone
8.5	Release infrastructure: offline signing, update server, SBOM, GPL source page	7.9	A signed beta is published with its source bundle
8.6	Closed beta with 10 to 20 testers; bug triage by severity	8.4	No open S1 or S2 bugs for two weeks

Gate: Public beta. Signed A/B updates with rollback, encrypted storage, all 17 stock apps, Android apps through the container, and every performance budget met.

First 12 weeks, week by week

Two tracks run side by side: the laptop track never waits for the phone, and the phone track fills the waiting time of the bootloader unlock.

Week	Laptop track	Phone track	Check at the end of the week
1	1.1 Monorepo, workspace, docs committed	Request unlock; back up; flash stock firmware	<code>cargo build</code> passes; unlock requested
2	1.2 CI; 1.4 toolchain	Wait for unlock; read the postmarketOS miatoll notes	CI green on a PR
3	2.1 Smithay window at 60 fps	1.5 Unlock, install base image with Phosh	Window with frame timer; phone boots
4	2.2 GLES renderer, damage tracking	1.6 Hardware checklist, baselines	STATUS.md complete
5	2.3 Liquid Aurora wallpaper	1.7 SSH and <code>sp logs</code> ; hello-world on phone	Wallpaper matches the prototype
6	2.4 Glass v0: blur	Report or fix any failing hardware upstream	One blurred panel
7	2.5 Glass v1: refraction, tint, rim, cache	Cross-compile compositor, run it on the phone under Phosh as a test	Glass matches the prototype
8	2.6 Spring engine with tests	Measure glass cost on Adreno 618	Springs match spec values
9	2.7 Gestures: swipe to unlock mock-up	Start 3.1 DRM/KMS backend	Swipe feels like the prototype
10	2.8 Show an external Wayland app	3.1 Wallpaper on the phone screen	External app renders inside
11	2.9 Debug overlay	3.2 Touch through libinput	F12 overlay; touch moves the panel
12	Buffer week: fix what slipped	Buffer week	Gates Bring-up done and Glass on laptop passed

The detailed tasks for weeks 1 to 4 are in `docs/process/backlog-milestone-1.md`.

Critical path and parallel tracks

The critical path runs through the phone, not the laptop: any delay in unlocking, bring-up, the DRM backend or telephony delays the whole project.

Critical path: bootloader unlock → 1.5 base image → 1.6 hardware checklist → 3.1 DRM backend → 3.5 compositor as the session → 4.11 telephony → 5.9 Phone app → **Usable phone** → 6.1 sandbox → 6.2 permissions → 7.3 container manager → 7.5 bridge → **Android apps** → 8.6 closed beta → **Public beta**.

Parallel tracks (work these whenever the critical path is waiting on hardware, upstream fixes or tests)

Track	Work packages	Can start once
Graphics and look	2.1 to 2.9, 3.4, 3.6	Workspace exists (1.1)
Toolkit	4.1 to 4.7	Workspace exists (1.1)
Stock apps on the laptop	6.6, built against the windowed compositor	Component library half done (4.4)
Signature features	6.7	Shell basics exist (5.3, 5.6)
Security groundwork	7.9, 7.10 prototypes on a spare board or in QEMU	sp tool exists (1.3)
Documentation and community	Demo videos, contributor guide updates	Glass on phone gate

Where a second contributor helps most: the toolkit and stock apps track, which is large, laptop-only and independent of the phone.

Rules for moving forward

Simple rules that keep a long solo project from drifting.

1. **Gates are hard.** Do not start the next dependent stage in earnest until its gate passes; small spikes are fine, full builds are not.
2. **Definition of done for every work package:** code merged to `main`, tests passing in CI, works on the phone (or on the laptop for laptop-only packages), and the matching doc updated.
3. **Weekly review (30 minutes):** tick finished work packages, move slipped ones, and write one line on what blocked you. If the same blocker appears three weeks running, change the plan, not your sleep.
4. **Buffer is planned, not borrowed:** every sixth week is a buffer week for slippage and bugs.

5. **Do not build yet:** v2 features (Desktop mode, Glass Lens, Offline Nearby, Private Memory, decoy data, domain firewall, Private Space, split screen) until after the public beta. Ideas go into a parking-lot issue.
6. **Upstream first:** kernel and packaging fixes go to the device maintainers and postmarketOS rather than being carried as private patches.
7. **Measure, do not guess:** any performance claim comes with a trace from the device.
8. **Protect the phone:** never touch the persist or modem partitions; keep the stock fastboot ROM downloaded and ready.